**UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO**

**FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES**




# Primer Parcial

# Proyecto: Ciudad y Conducción con OpenGL

**Materia:** Programación Gráfica

**Sigla:** ELC102 - SA

**Docente:** Ing. Kenji Kawaida Villegas

**Estudiante:** Sara Eunice Navarro Acosta

**Semestre II/2026**

**Santa Cruz de la Sierra – Bolivia**

**1. Descripción general**
Proyecto didáctico en Java 17 + Maven + LWJGL 3.3.3 con GLFW y OpenGL 3.3 Core + shaders GLSL, VAO y VBO. Toda la geometría se genera por código con un cubo unitario transformado (caja(), cajaGirada(), pieza()). No usa imágenes, modelos, texturas ni librerías de texto.
Estructura acumulativa por herencia, cada clase tiene su propio main:
clase1 -> clase2 -> clase3 -> clase4 -> clase5 -> clase6 -> clase7 -> clase8 -> clase9 -> clase10 -> clase11

**2. Diseño de la ciudad**
- Matriz MAPA de 11x11 en clase1.java:85 (ya no está en la 82). 0=calle, 1=edificio, 2=parque, 3=hospital. Las filas corresponden a Z y las columnas a X; Y es altura.
- Cada celda mide 10 unidades (CELDA, clase1.java:82). Mapa de 110x110, LIMITE=55. centro(indice) convierte índices de matriz a coordenadas del mundo.
- Calles en retícula conectada (filas/columnas pares, índices 0,2,4,6,8,10); manzanas 5x5 intercaladas en los cruces de índices impares. Cada calle mide 10 unidades y los cruces quedan despejados.
- Base: plataforma de asfalto 110x0.5x110 con su cara superior en Y=0 (color gris oscuro 0.16/0.19/0.23). Cada manzana lleva una acera de 10x0.3x10 elevada a Y=0.15.
- Marcas viales: líneas amarillas discontinuas (1, 0.84, 0.35) solo en celdas de calle, 3 tramos por celda separados 3 unidades; no se dibujan sobre los cruces. En el minimapa se engrosan y elevan (0.7 y 0.15 contra 0.13 y 0.03) para que se vean en proyección ortográfica (clase1.java:304-332).
- Edificios: cuerpo 7x7 de altura variable 5 + (fila*3 + columna*7) % 9 (entre 5 y 13), con color que depende de fila/columna, y cubierta 7.3x0.3 ligeramente más ancha y oscura a modo de techo (clase1.java:277-285).
- Hospitales: cuerpo blanco 7x7x4.5 fijo, cubierta blanca y cruz roja de dos cajas sobre el techo (clase1.java:286-292).
- Parques: césped verde 9x9, 4 árboles (tronco 0.35x2 + copa 2x2.3) en las diagonales +/-2.5, y 1 banco centrado con asiento, respaldo y soporte (clase4.java:154-167).
- Ventanas iluminadas: solo de noche y con emisión (no las oscurece la iluminación); 3 por fachada en las 4 fachadas, un piso cada 2 unidades de altura desde Y=1.7 (clase4.java:170-188).
- Pasos peatonales: 7 franjas blancas paralelas sobre la calle, al norte de cada manzana (z-8), con glPolygonOffset para evitar z-fighting con el asfalto (clase4.java:193-201).
- Semáforos animados: poste + carcasa de 3 luces en la esquina NE de cada manzana (x+4, z-4). Ciclo de 12 s: rojo 5 s, verde 5 s, amarillo 2 s; las bombillas apagadas se quedan a brillo 0.15 (clase4.java:207-239). Son decorativos: no detienen al tráfico (por eso el reloj tiempo nunca se congela).
- Farolas: 25 en total (una por manzana, no 9), en la esquina NO de cada manzana (x-4, z-4). Poste de 4.4 y bombilla a Y=4.5; de noche emiten en cálido y su luz se envía al shader en uLuces[25] (clase3.java:58-73 y 102-117).

- Iluminación: ambiente + sol direccional + atenuación por distancia + conos de faros del auto. Día: ambiente 0.48 y sol 0.65; noche: ambiente tenue azulado y sol 0.10. Sin sombras: la luz atraviesa los edificios (clase3.java:126-189).
- Baliza dorada del destino: plataforma dorada 5x5 sobre el asfalto más un cubo giratorio flotante que oscila con seno (3.5 + sin(tiempo*2)*0.3). Tres entregas en las esquinas (50,-50), (-50,-50) y (-50,50); se completan a menos de 3 unidades y a menos de 1 u/s (clase4.java:22-26 y 115-126). La baliza 3D se omite en el minimapa.
- Minimapa (arriba-derecha, clase4): segundo pase de dibujo con viewport y scissor propios (no borra la escena), vista ortográfica con el norte (-Z) arriba, hasta 260 px o 1/3 de la ventana, con marco claro. Omite la decoración pequeña y la baliza; marca al jugador en cian con punta blanca, a los autos del tráfico en gris claro y a los peatones con puntos naranjas. Tecla M para ocultarlo (clase4.java:248-306, clase5.java:546-547, clase8.java:347-352).
- Panel (abajo-derecha, clase6): recuadro de hasta 230 px o 1/3 de la ventana, oculto si la ventana es muy pequeña (<120 px de lado). Tres filas: VEL con km/h y barra proporcional (0-16 m/s, de verde a rojo), FAROS ON/OFF y HORA DÍA/NOCHE, con lámpara indicadora. El texto ya no es fuente de puntos: usa fuente real del sistema con suavizado, pegada como textura (Texto2D). Desde clase6 el título de la ventana ya no repite la velocidad (clase6.java:28-92).
- Menú central al inicio (clase7): recuadro centrado con DELIVERY EN LA CIUDAD, "COMENZAR -> W o ^" parpadeando cada segundo y "SALIR -> ESC". Mientras está abierto, el auto, las entregas y el tráfico quedan detenidos, pero la cámara orbita sola (0.15 rad/s) y el reloj avanza, así que los semáforos y el parpadeo siguen animándose de fondo (clase7.java:55-68 y 136-153).
- Peatones (clase8): 3 personas caminando por las aceras (a 4.2 del centro de manzana) que cruzan por los pasos peatonales, con piernas y brazos balanceándose al paso. Frenan si un auto está sobre el paso o muy cerca (radios 2.8 en acera, 5.0 para entrar a la calle, 2.0 ya cruzando); tecla P para pausarlos.
- Tráfico (clase5): 3 autos blancos autónomos en circuitos concéntricos sobre las calles, siempre por la derecha del eje, que giran en las esquinas y ceden el paso; tecla T para encenderlos/apagarlos.
- Todo se genera con código reutilizando un único cubo unitario (VAO/VBO compartidos); no hay modelos ni imágenes externas.

**3. Controles**

| **Tecla** | **Acción** |
|---|---|
| 1 / 2 | Orbitar ciudad |
| W / ↑ , S / ↓ | Acelerar / retroceder |
| A / D o ← / → | Girar (proporcional a velocidad, invierte en reversa) |
| Espacio | Freno |

| | |
|---|---|
| C | Cámara seguimiento / aérea oblicua |
| R | Reiniciar auto (+ entregas y tráfico) |
| N | Día / noche |
| F | Faros ON/OFF |
| M | Mostrar/ocultar minimapa |
| T | Seguir/ parar tráfico |
| W / ↑ | Salir del menú inicial |
| ESC | Salir |
| P | Pausar / reanudar peatones |

**4. Funciones implementadas**

- Clase 1: Ciudad y cámara: mapa, conversión matriz→mundo centro(), cubo de 36 vértices con normales, shaders perspectiva/ortográfica (uMapa, uLimite), ciclo run/iniciar/loop/dibujarFrame/limpiar.
- Clase 2: Auto y conducción: auto rojo (chasis, cabina, ruedas, faros), física con aceleración/resistencia/freno/límites, dirección con seno/coseno, colisión círculo-vs-rectángulo + borde (puedeCircular()), 2 cámaras.
- Clase 3: Iluminación: modo día/noche, 9 farolas punto + faros cónicos, bombillas con emisión, uniforms uNoche, uFaros, uAuto.
- Clase 4: Ciudad final + juego: decoración, 3 entregas (DESTINOS, completar a <3u y <1 u/s, GANASTE + R), semáforos, minimapa norte (-Z) arriba con auto cian + destino dorado.
- Clase 5: Tráfico autónomo: 3 autos blancos en circuitos cerrados concéntricos a la derecha (RUTA_EXTERNA/MEDIA/CENTRAL, CARRIL=2.4), IA por tramos con punto de mira MIRA, frenado en cruces, evasión lateral, anti-colisión con rectángulos orientados (SAT) + puedeCircular().
- Clase 6: Panel HUD: recuadro inferior-derecho con VEL xx KM/H + barra 0-16 m/s (verde→rojo), FAROS ON/OFF y HORA DIA/NOCHE con lámparas. Fuente propia 5x7 dibujada con cajas (glifo()).
- Clase 7: Menú inicio: recuadro central DELIVERY EN LA CIUDAD / COMENZAR -> W o ^ / SALIR -> ESC, bloquea conducción/tráfico hasta pulsar W, fondo con órbita lenta.
- Clase 8 — Peatones: 3 personas con rutas por las aceras (a 4.2 del centro de manzana) que cruzan por los pasos peatonales; balanceo de brazos y piernas según lo caminado (FRECUENCIA_PASO=6); esperan si un auto está sobre el paso o muy cerca (radios 2.8 en acera, 5.0 para entrar a la calle, 2.0 ya cruzando); P pausa/reanuda y R los devuelve a su esquina; puntos naranjas en el minimapa.

- Clase 9 — Retrovisores: dos espejos laterales (brazo oscuro, carcasa del color del auto y vidrio claro) añadidos dentro de dibujarCarroceria(), así el auto rojo y los 3 blancos del tráfico los reciben sin duplicar medidas.
- Clase 10 — Autos más grandes: factor ESCALA_AUTOS=1.3 aplicado con piezaGrandeEn() a todas las medidas locales (posición y tamaño) de los 4 autos, para que crezcan parejo sin deformarse.
- Clase 11 — Chata (remolque) articulada: remolque de carga unido solo al auto del jugador por la bocha de enganche (a 1.74 del centro); pivota con su propio anguloChata (máx. 25° de articulación, amortiguación 0.35), recorta curvas al avanzar y se tijerea en reversa; colisiones por puntos (lanza, centro y parte trasera): si la chata choca o se atasca, el auto no avanza hacia el obstáculo (velocidad 0) y hay que maniobrar; permite girar el volante en parado (<0.6 u/s) para desatascarse; construcción con lanza triangular, chasis de acero, piso y barandas de madera, arquillos tubulares, doble eje con 4 ruedas, guardabarros y luces traseras emisivas; cámara de seguimiento ajustada (14.5 detrás, altura 10) y arranque en autoZ=43 para dejar sitio al conjunto.
- Texto2D (clase auxiliar de las clases 6 y 7): escribe texto con fuente real del sistema (sans serif negrita) sobre un BufferedImage con suavizado, al doble de resolución, y lo sube a la GPU como textura con mipmaps; solo regenera la textura cuando el texto cambia; reemplaza a las letras pixeladas hechas con cajas.

**5. Limitaciones**

- Sin audio, sombras ni oclusión: las farolas y los faros del auto iluminan a través de paredes y edificios (no hay shadow maps); tampoco hay reflejos: los espejos de la clase 9 son solo geometría.
- Cielo fijo: el fondo no cambia con N; día/noche solo afecta superficie y luces. Todo es geometría generada por código (cajas), sin texturas ni modelos importados.
- Semáforos decorativos: ni el tráfico ni los peatones los obedecen; solo cambian de color cada 12 s (5 rojo, 5 verde, 2 amarillo).
- Conducción: el choque detiene sin rebotes; no hay ruedas que giren ni animación de volante; el auto no colisiona con los peatones (la persona se aparta sola, pero el auto puede atravesarla). La colisión del jugador sigue siendo un círculo conservador (1.65), aunque tras el agrandado 1.3× de la clase 10 queda algo justo respecto al largo visible del auto.
- Tráfico: no retrocede ni acelera en las curvas, no cambia de circuito ni de carril (solo evasión lateral) y espera si el jugador bloquea el paso. Sus medidas de colisión (MEDIO_LARGO/MEDIO_ANCHO) tampoco se re-escalaron con el agrandado, así que puede notarse un roce visual.
- Ni el tráfico ni los peatones ven la chata: no la evitan; si un auto del tráfico la toca, el conjunto del jugador se detiene y hay que maniobrar.
- Peatones: solo 3, con rutas fijas por aceras; cruzan guiándose por los autos (no por el semáforo), giran en seco en las esquinas y no hay colisiones entre personas ni más animaciones que el balanceo de brazos y piernas, y no hay mujeres.
- Chata (clase 11): solo existe para el auto del jugador y no se puede desenganchar; articulación limitada a 25°; la cámara de seguimiento fija detrás (14.5, altura 10) no orbita

alrededor del remolque, y el minimapa marca el auto pero no tiene marca propia para la chata. Las entregas se completan siempre con el remolque puesto.
- HUD, menú y minimapa: panel y menú se ocultan si la ventana es muy pequeña; el minimapa es fijo (norte -Z arriba, sin rotación ni zoom) y limitado a 260 px o 1/3 de la ventana. El texto usa fuentes AWT del sistema (Texto2D), no la iluminación de la escena.
- Partida: 3 destinos fijos en las esquinas (no aleatorios); el cronómetro se congela al ganar, pero el reloj de los semáforos no.
- Requisitos técnicos: necesita sesión gráfica con OpenGL 3.3; no funciona headless. Con JDK 25 aparecen avisos de acceso nativo, Unsafe y JNI; se recomienda JDK 17.

**6. Instrucciones de ejecución**

Requisitos: JDK 17+, Maven, GPU con OpenGL 3.3.
En la carpeta donde está pom.xml:

```
mvn compile exec:exec "-DmainClass=com.graphics.clase1"
mvn compile exec:exec "-DmainClass=com.graphics.clase2"
mvn compile exec:exec "-DmainClass=com.graphics.clase3"
mvn compile exec:exec "-DmainClass=com.graphics.clase4"
mvn compile exec:exec "-DmainClass=com.graphics.clase5"
mvn compile exec:exec "-DmainClass=com.graphics.clase6"
mvn compile exec:exec "-DmainClass=com.graphics.clase7"
mvn compile exec:exec "-DmainClass=com.graphics.clase8"
mvn compile exec:exec "-DmainClass=com.graphics.clase9"
mvn compile exec:exec "-DmainClass=com.graphics.clase10"
mvn compile exec:exec "-DmainClass=com.graphics.clase11"
```

Sin -DmainClass arranca clase4 por defecto. En macOS se agrega -XstartOnFirstThread automáticamente. Pruebas lógicas sin ventana: mvn test.

**7. Recursos y fragmentos externos utilizados**

Ningún modelo, textura, imagen, fuente ni audio externo: no hay archivos .png, .jpg, .ttf, .wav, .obj, etc. en el proyecto; toda la geometría se genera con código propio. Lo que sí se usa:
- org.lwjgl:lwjgl, lwjgl-glfw, lwjgl-opengl:3.3.3 + nativos (Windows x64, Linux x64/ARM64 y macOS Intel/Apple Silicon) — ventana, entrada y OpenGL.
- org.lwjgl BufferUtils — buffer nativo para subir la textura del texto a la GPU (Texto2D).
- org.codehaus.mojo:exec-maven-plugin:3.1.0 y maven-compiler-plugin:3.13.0 — compilación y ejecución (mvn compile exec:exec). El surefire de las pruebas lo aporta Maven por defecto (no está fijado en el pom.xml).
- junit:junit:3.8.1 (estilo JUnit 3, TestCase) — solo en las pruebas ConduccionTest y TraficitoTest.
- Biblioteca estándar de Java (JDK, no dependencia externa): java.util (HashMap/Map en clase1, ArrayList en clase3), y java.awt + java.nio en Texto2D, clase6 y clase7.
- Texto del panel y del menú: reemplaza a la antigua fuente 5x7 dibujada a mano con cajas y matrices "X/." Ahora las letras usan una fuente real del sistema (Font.SANS_SERIF en negrita, Texto2D.java:187) dibujada con java.awt.Graphics2D sobre un BufferedImage con suavizado, al doble de resolución (SUPER=2, Texto2D.java:57) y subida a la GPU como textura con

mipmaps; solo se regenera cuando el texto cambia. No se incluye ningún archivo de fuente: la tipografía la aporta el sistema operativo.
- Base docente: AppCamara.java y AppLaberinto.java de clase (citada en README.md) — estructura de ventana/contexto/shaders y cubo unitario de 36 vértices, reutilizada y ampliada para ciudad, conducción, iluminación, tráfico, peatones y remolque.
- Técnicas estándar implementadas a mano (no copiadas de tutoriales): iluminación Lambert con atenuación cuadrática, cono de luz con smoothstep, colisión círculo-vs-rectángulo, SAT para rectángulos orientados y circuitos cerrados de waypoints.